

Data Preprocessing & Data Handling

The Complete Guide — From Raw Data to Model-Ready Features

"Garbage in, garbage out." No algorithm, however sophisticated, can rescue a model trained on poorly prepared data. Data preprocessing is not a formality before the "real" ML work — **it IS the real work**. In industry, data scientists spend 60–80% of their time here.

How to Use This Guide

This guide follows a **single real-world dataset throughout every section** so you always see concepts applied, not just defined.

 **Running Example — House Price Prediction** You have a dataset of 10,000 houses with features:

Age | Area_sqft | Bedrooms | Location | Furnishing | Distance_to_Metro | Price | Last_Renovated | Owner_Income

Your goal: Predict **Price** (regression) or **Expensive: Yes/No** (classification). This dataset will have missing values, outliers, categorical variables, skewed distributions, irrelevant features — everything you will face in the real world.

PART 1 — Understanding Your Data Before Touching It

1.1 Why You Must Explore Before You Clean

The single most common mistake beginners make is jumping straight into cleaning without understanding what the data actually looks like. Every preprocessing decision — how to handle missing values, which outliers to remove, which features to engineer — depends on understanding the data's structure first.

The first questions to ask about any dataset:

1. How many rows and columns?
2. What is the data type of each column?
3. How many missing values per column, and what percentage?
4. What does the distribution of each numerical column look like?
5. What are the unique values in categorical columns?
6. Are there obvious data entry errors?
7. What is the relationship between features and the target?

In Python:

```
import pandas as pd
import numpy as np
```

```

df = pd.read_csv('houses.csv')

# Shape
print(df.shape)           # (10000, 9)

# Data types and non-null counts
print(df.info())

# Statistical summary of numerical columns
print(df.describe())

# Missing value count and percentage
missing = df.isnull().sum()
missing_pct = (df.isnull().sum() / len(df)) * 100
print(pd.DataFrame({'Count': missing, 'Percentage': missing_pct}))

# Unique values in categorical columns
print(df['Location'].value_counts())
print(df['Furnishing'].unique())

```

Sample output of `df.describe()` for our dataset:

	Age	Area_sqft	Bedrooms	Price
count	9,200	9,750	10,000	9,800
mean	18.4	1,450	2.8	45,20,000
min	0	200	1	8,00,000
max	95	12,000	18	8,50,00,000
std	12.1	890	1.4	32,50,000

Already you can see: **Age** has 800 missing values (10%), **Bedrooms = 18** looks like a data entry error, the price range is enormous suggesting outliers.

1.2 Data Types — Know What You're Working With

Every preprocessing decision flows from understanding what kind of data each column contains.

NUMERICAL (Quantitative)

- |
 - Continuous : Can take any value in a range
 Examples: Price, Area_sqft, Distance_to_Metro
 Operations: mean, median, std, scaling
 - Discrete : Only integer values
 Examples: Bedrooms, Age (in years), Floor number
 Operations: same as continuous, but be careful with scaling

CATEGORICAL (Qualitative)

- Nominal : Categories with NO natural order
Examples: Location (Andheri/Bandra/Dadar), Furnishing (Yes/No)
Operations: One-Hot Encoding, Target Encoding
- Ordinal : Categories WITH a natural order
Examples: Condition (Poor/Average/Good/Excellent)
Operations: Label Encoding (with care), Ordinal Encoding

DATETIME

Examples: Last_Renovated (2018-03-15)
Operations: Extract Year, Month, Days_since, Season
NEVER feed raw datetime strings to a model

TEXT

Examples: Property Description, Owner Comments
Operations: TF-IDF, Word Embeddings, NLP pipeline

PART 2 — Handling Missing Values

2.1 Why Missing Values Exist — The Three Types

Not all missing values are equal. The *reason* data is missing determines how you should handle it.

MCAR — Missing Completely At Random

The missingness has **no relationship** to any variable in the dataset. Pure chance — a sensor glitch, a form submission that dropped a field.

Example: 5% of houses are missing **Age** simply because the data entry form had a bug on certain days.

Implication: Safe to impute or drop rows without introducing bias.

MAR — Missing At Random

The missingness is related to **other observed variables** but not to the missing variable itself.

Example: Houses in rural locations are more likely to have missing **Distance_to_Metro** — because they don't have metros nearby, not because of anything about the distance value itself.

Implication: Can be safely imputed using other columns as context. Simply dropping rows would bias your dataset toward urban properties.

MNAR — Missing Not At Random

The missingness is directly related to the **value that is missing**. This is the most dangerous type.

Example: Very expensive houses (Price > 5 Crore) have missing `Owner_Income` because wealthy owners deliberately didn't disclose it.

Implication: The missing data itself is informative. Imputing without acknowledging this introduces bias. You may need to create a "Was_Missing" flag feature.

2.2 Detecting Missing Values

```
# Count missing per column
df.isnull().sum()

# Percentage missing
(df.isnull().sum() / len(df) * 100).round(2)

# Heatmap of missingness pattern (see which columns are missing together)
import seaborn as sns
import matplotlib.pyplot as plt
sns.heatmap(df.isnull(), cbar=False, yticklabels=False)
plt.show()

# Check if missingness in one column correlates with another
print(df[df['Owner_Income'].isnull()][ 'Price'].describe())
# If the Price stats here look different from the overall Price stats → MNAR
```

2.3 Strategy 1 — Dropping

When to drop rows:

```
df.dropna(subset=[ 'Price' ])          # Drop rows where target is missing (always
do this)
df.dropna(thresh=int(0.5 * df.shape[1])) # Drop rows with >50% columns missing
```

When it's safe: MCAR data, very few rows affected (<1–2%), or when missing values are in the target column.

When it's dangerous: MAR/MNAR data. Dropping rows selectively removes certain types of houses, introducing sampling bias. Your model will perform worse on exactly those cases.

When to drop columns:

```
df.drop(columns=[ 'Owner_Income' ])    # Drop a column
```

Rule of thumb: If a column has **>40–50% missing** and you cannot find a logical imputation strategy, dropping is safer than imputing noise.

2.4 Strategy 2 — Simple Imputation (Statistical)

Mean Imputation

```
x_imputed = mean of non-missing values in the column
```

```
from sklearn.impute import SimpleImputer

imputer = SimpleImputer(strategy='mean')
df['Area_sqft'] = imputer.fit_transform(df[['Area_sqft']])
```

Use when: Data is roughly normally distributed, MCAR, and the missing percentage is low (<5%).

Do NOT use when: Data has outliers (the mean is pulled toward outliers, imputing wrong values for most patients) or when data is skewed.

Effect on distribution: Mean imputation reduces variance (all imputed values are the same number) and distorts correlations between features.

Median Imputation

```
x_imputed = median of non-missing values in the column
```

```
imputer = SimpleImputer(strategy='median')
df['Age'] = imputer.fit_transform(df[['Age']])
```

Use when: Data is **skewed** or has outliers. Median is robust — it is not pulled by extreme values.

Example: Age of houses. Most houses are 5–25 years old, but a few heritage properties are 80–95 years old. Mean age might be pulled to 22 years while median is 14 years. Imputing with 14 years is more representative.

Mode Imputation

```
x_imputed = most frequently occurring value in the column
```

```
imputer = SimpleImputer(strategy='most_frequent')
df['Furnishing'] = imputer.fit_transform(df[['Furnishing']])
```

Use when: Categorical columns with missing values. Mean and median are meaningless for categories.

Caution: If one category dominates (70% of houses are "Unfurnished"), mode imputation amplifies this majority — potentially masking the predictive signal of minority categories.

2.5 Strategy 3 — Advanced Imputation

KNN Imputation

The idea: find the K most similar houses (based on other features) to the house with a missing value, and use their average to fill in the gap.

$$x_{\text{imputed}} = (1/K) * \text{Sum}(\text{values from K nearest neighbors})$$

```
from sklearn.impute import KNNImputer

imputer = KNNImputer(n_neighbors=5)
df_imputed = pd.DataFrame(
    imputer.fit_transform(df),
    columns=df.columns
)
```

Why it's better than mean/median: It uses the relationship between features. A 3-bedroom house in Bandra with Area=1200 sqft that is missing Age will be imputed using similar 3-bedroom Bandra houses around 1200 sqft — not the average age of all 10,000 houses.

Requirement: All features passed to KNN Imputer must be numerical (encode categoricals first).

Cost: Computationally expensive for large datasets.

Iterative Imputation (MICE — Multiple Imputation by Chained Equations)

The idea: treat each column with missing values as the **target** and use all other columns as features to predict it. Repeat this process iteratively until the imputed values converge.

```
from sklearn.experimental import enable_iterative_imputer
from sklearn.impute import IterativeImputer

imputer = IterativeImputer(max_iter=10, random_state=42)
df_imputed = pd.DataFrame(
    imputer.fit_transform(df),
    columns=df.columns
)
```

Why it's powerful: It captures complex multivariate relationships. To impute **Age**, it builds a regression model using **Area_sqft**, **Bedrooms**, **Price**, **Location** etc. as predictors.

When to use: MAR data, where missingness is related to other observed features. Gold standard for imputation in research settings.

2.6 Strategy 4 — Creating a "Was_Missing" Flag (For MNAR)

When data is MNAR, the fact that a value is missing is itself informative. Create a binary indicator column before imputing.

```
# Create flag BEFORE imputing
df['Owner_Income_Missing'] = df['Owner_Income'].isnull().astype(int)

# Now impute (with any strategy)
df['Owner_Income'].fillna(df['Owner_Income'].median(), inplace=True)
```

Why this matters: Now the model learns two things:

1. The actual Owner_Income value (imputed)
 2. Whether that value was originally missing (which itself predicts price, since wealthy owners hid income)
-

2.7 The Golden Rules of Missing Value Handling

RULE 1: Always investigate WHY values are missing before deciding HOW to handle them.

MCAR → simple imputation or drop
MAR → KNN or iterative imputation
MNAR → flag + impute, or domain-driven imputation

RULE 2: Fit the imputer on TRAINING data only.
Apply (transform) on both train and test.
NEVER fit on test data – that leaks test information.

RULE 3: Handle the target column first.
Rows where the target (Price) is missing must always be dropped.
You cannot impute what you are trying to predict.

RULE 4: Document every imputation decision.
"I imputed Age with median because it was right-skewed (skewness=2.1) and only 8% of values were missing (MCAR)."

RULE 5: Validate that imputation didn't destroy relationships.
Check correlation before and after imputation.
Check distribution before and after imputation.

PART 3 — Handling Outliers

3.1 What Is an Outlier and Why Does It Matter?

An outlier is a data point that is **significantly different from the rest** of the distribution. But "significantly different" has two interpretations:

Statistical outlier: A point that is far from the mean in terms of standard deviations. May be a measurement error.

Domain outlier: A point that is statistically unusual but genuinely valid.

Example in our dataset:

- **Bedrooms = 18** — almost certainly a data entry error (should be 1.8 or 1-8).
- **Price = 8,50,00,000** — this might genuinely be a luxury penthouse in South Mumbai. Removing it would mean your model never learns from high-value properties.

The impact of outliers on models:

Model	Sensitivity to Outliers
Linear Regression	Very HIGH — MSE squares errors, so one massive outlier dominates the gradient
Logistic Regression	Moderate — boundary can be pulled toward outlier cluster
KNN	Moderate — outliers become "lonely" neighbors that mislead nearby predictions
SVM (Hard Margin)	Very HIGH — one outlier point can completely destroy the margin
SVM (Soft Margin)	LOW — slack variables absorb outlier violations
Decision Tree / Random Forest	LOW — tree splits are based on ordering, not magnitude
Neural Networks	HIGH — unless batch normalisation or robust loss functions are used

3.2 Detecting Outliers

Method 1 — Z-Score (For Normally Distributed Data)

The Z-score measures how many standard deviations a point is from the mean.

```
Z_i = (x_i - mean) / std_deviation
```

```
from scipy import stats
```

```
z_scores = np.abs(stats.zscore(df['Price']))
outliers_z = df[z_scores > 3]          # Points more than 3 std devs from mean
print(f"Outliers detected: {len(outliers_z)}")
```

Threshold: $|Z| > 3$ is the standard rule (99.7% of normally distributed data falls within ± 3 std devs).

Limitation: Z-score itself uses the mean and std, which are already influenced by outliers. For heavily skewed data, use IQR instead.

Method 2 — IQR (Interquartile Range) — Robust to Skew

$IQR = Q3 - Q1$

Lower Fence = $Q1 - 1.5 * IQR$

Upper Fence = $Q3 + 1.5 * IQR$

Any point outside these fences is flagged as an outlier.

```
Q1 = df['Price'].quantile(0.25)
Q3 = df['Price'].quantile(0.75)
IQR = Q3 - Q1

lower_fence = Q1 - 1.5 * IQR
upper_fence = Q3 + 1.5 * IQR

outliers_iqr = df[(df['Price'] < lower_fence) | (df['Price'] > upper_fence)]
print(f"Outliers detected: {len(outliers_iqr)}")
```

Why 1.5? Tukey's rule: $1.5 \times IQR$ captures ~99.3% of normally distributed data. Use **$3 \times IQR$** for "extreme" outliers only.

IQR is preferred for skewed data (like Price, Income, Area) because it is based on quantiles, which are rank-based and unaffected by extreme values.

Method 3 — Visual Detection

```
import matplotlib.pyplot as plt
import seaborn as sns

# Box plot — shows IQR, whiskers, and outlier points
sns.boxplot(x=df['Price'])
plt.show()

# Histogram — shows distribution shape and outlier gaps
df['Price'].hist(bins=50)
```

```
plt.show()

# Scatter plot – shows outliers in 2D context
plt.scatter(df['Area_sqft'], df['Price'], alpha=0.3)
plt.xlabel('Area')
plt.ylabel('Price')
plt.show()
```

3.3 Handling Outliers — The Four Strategies

Strategy 1 — Remove (Drop the Row)

```
df_clean = df[(df['Price'] >= lower_fence) & (df['Price'] <= upper_fence)]
```

When to use: Confirmed data entry errors (Bedrooms=18), measurements that are physically impossible (Age=-5), or genuine noise in sensor data.

When NOT to use: When outliers are real — a genuine luxury penthouse is important training data for predicting luxury house prices. Removing it means your model is blind to that segment.

Strategy 2 — Capping / Winsorisation

Replace outlier values with the fence values instead of removing the row entirely.

```
lower_fence = df['Price'].quantile(0.01) # 1st percentile
upper_fence = df['Price'].quantile(0.99) # 99th percentile

df['Price_capped'] = df['Price'].clip(lower=lower_fence, upper=upper_fence)
```

Why it's often better than dropping: You retain all rows (important for sample size). The extreme values are brought to the boundary of "reasonable" rather than deleted. The row still contributes all its other features.

Common thresholds: 1st–99th percentile (aggressive capping) or 5th–95th percentile (conservative).

Strategy 3 — Log Transformation (For Right-Skewed Data)

When a feature like Price has a long right tail (a few extremely high values), taking the log compresses the scale and makes the distribution more symmetric.

```
x_transformed = log(x)           or   log(x + 1)  if x can be 0
```

```
df['Price_log'] = np.log1p(df['Price'])      # log(Price + 1)
df['Area_log']  = np.log1p(df['Area_sqft'])
```

Before: Price distribution is heavily right-skewed (skewness = 4.2). A few houses at 8 Crore pull the mean far from the median.

After: Log(Price) is approximately normal (skewness ≈ 0.3). Linear models perform much better on the transformed target.

Important: If you log-transform the target variable (Price), remember to **reverse the transformation** (exponentiate) when interpreting predictions.

```
# Predicted log(Price) = 15.3
predicted_price = np.exp(15.3)      # e^15.3 - 1
```

Strategy 4 — Treat Separately (Segmentation)

Sometimes outliers represent a genuinely different population. Build a separate model for them.

Example: Split the housing dataset into:

- Segment A: Price < 1 Crore (middle market) → one model
- Segment B: Price >= 1 Crore (luxury market) → separate model

This is common in finance (retail vs. institutional clients) and healthcare (typical patients vs. rare disease cases).

3.4 The Key Decision Framework

```
Is this outlier a genuine data error?
|
├── YES (Bedrooms=18, Age=-5)
│   └── Remove the row or correct the value if you know the true value
└── NO (real luxury penthouse, genuinely wealthy buyer)
    ├── Is your model sensitive to outliers? (Linear Regression, SVM hard margin)
    │   └── YES → Cap/Winsorise OR Log-transform the feature
    └── Is your model robust to outliers? (Tree-based, SVM soft margin)
        └── NO action needed – the model handles it internally
```

PART 4 — Encoding Categorical Variables

4.1 Why Models Cannot Eat Categories Directly

Every ML algorithm operates on numbers — matrix multiplications, distance calculations, gradient computations. A category like "Bandra" has no mathematical meaning. You must convert it into numbers in a way that **preserves the information** without introducing false relationships.

Our dataset's categorical columns:

```
Location    → Nominal (Andheri, Bandra, Dadar, Kurla, Thane, ...) — 12 unique values
Furnishing  → Nominal (Furnished, Semi-Furnished, Unfurnished) — 3 unique values
Condition   → Ordinal (Poor, Average, Good, Excellent) — 4 levels with order
```

4.2 Label Encoding

Assigns a unique integer to each category.

```
Furnished      → 0
Semi-Furnished → 1
Unfurnished    → 2
```

```
from sklearn.preprocessing import LabelEncoder

le = LabelEncoder()
df['Furnishing_encoded'] = le.fit_transform(df['Furnishing'])
```

ONLY use for: Ordinal categories (where the order is meaningful) or tree-based models.

NEVER use for nominal categories with non-tree models: Label encoding implies **Unfurnished (2) > Semi-Furnished (1) > Furnished (0)**. Linear Regression would interpret "Unfurnished" as twice as much of something as "Semi-Furnished" — completely false.

4.3 One-Hot Encoding (OHE)

Creates a new binary (0/1) column for each unique category.

```
Location = Bandra
      ↓
Location_Andheri = 0
Location_Bandra  = 1      ← the one hot
Location_Dadar   = 0
```

```
Location_Kurla    = 0
...
```

```
df_ohe = pd.get_dummies(df, columns=['Location', 'Furnishing'], drop_first=True)
# drop_first=True removes one column to avoid multicollinearity (dummy variable trap)
```

Why `drop_first=True`? With 3 Furnishing categories, you only need 2 columns — if both `Furnished=0` and `Semi-Furnished=0`, the model already knows it's `Unfurnished`. Keeping all 3 creates **perfect multicollinearity** which breaks linear models.

Advantage: No false ordinal relationship. Each category is treated independently.

Disadvantage — The Curse of High Cardinality: If `Location` has 500 unique values (all neighbourhoods in Mumbai), OHE creates 499 new columns. This:

- Dramatically increases dimensionality (curse of dimensionality)
- Creates very sparse matrices (mostly zeros)
- Leads to many columns with very few 1s (rare categories)

Rule of thumb: Use OHE for categorical columns with **fewer than 10–15 unique values**.

4.4 Ordinal Encoding

For genuinely ordinal categories, manually define the mapping to preserve the order.

```
Poor      → 1
Average   → 2
Good      → 3
Excellent → 4
```

```
from sklearn.preprocessing import OrdinalEncoder

condition_order = [['Poor', 'Average', 'Good', 'Excellent']]
oe = OrdinalEncoder(categories=condition_order)
df['Condition_encoded'] = oe.fit_transform(df[['Condition']])
```

Why not just label encode? `OrdinalEncoder` lets you **specify the exact order**. `LabelEncoder` assigns integers alphabetically — which would give `Average=0`, `Excellent=1`, `Good=2`, `Poor=3` — completely wrong ordering.

4.5 Target Encoding (Mean Encoding)

Replace each category with the **mean of the target variable** for that category.

Location	Mean_Price
Bandra →	85,00,000
Andheri →	52,00,000
Dadar →	61,00,000
Thane →	38,00,000

```
# Calculate mean price per location
location_means = df.groupby('Location')['Price'].mean()

# Map back to the dataframe
df['Location_target_encoded'] = df['Location'].map(location_means)
```

Why it's powerful for high-cardinality features: Instead of 499 OHE columns for Location, you get exactly **1 column** that directly encodes each location's average price signal.

The Danger — Target Leakage: If you calculate mean price using ALL rows including the test set, you are leaking target information into features. Always compute the encoding on **training data only**.

The Danger — Overfitting on rare categories: A location with only 2 houses might have a mean price of 1.2 Crore simply because those 2 houses happened to be expensive. This is not a stable estimate.

Fix — Smoothing (Regularised Target Encoding):

$$\text{Smoothed_Mean} = (\text{count} * \text{category_mean} + m * \text{global_mean}) / (\text{count} + m)$$

where m is a smoothing parameter (e.g., 10). If a category has very few samples, its encoded value gets pulled toward the global mean — preventing overfitting on rare categories.

```
from category_encoders import TargetEncoder
encoder = TargetEncoder(smoothing=10)
df['Location_te'] = encoder.fit_transform(df['Location'], df['Price'])
```

4.6 Frequency / Count Encoding

Replace each category with how often it appears in the dataset.

Location	Count
Bandra →	1,250

```
Andheri → 2,100
Thane   → 3,400
```

```
freq_map = df['Location'].value_counts()
df['Location_freq'] = df['Location'].map(freq_map)
```

Use when: The frequency of a category is itself informative (e.g., in fraud detection, rare transaction types may signal fraud).

4.7 Encoding Strategy Summary

Situation	Best Encoding
Nominal, few categories (<15), tree or linear model	One-Hot Encoding
Ordinal category (clear order)	Ordinal Encoding with manual order
High cardinality (> 15 unique values), regression	Target Encoding with smoothing
High cardinality, any model, frequency matters	Frequency Encoding
Tree-based model only	Label Encoding is acceptable
Binary category (Yes/No, Male/Female)	Label Encoding (0/1) is fine

PART 5 — Feature Scaling

5.1 Why Scaling Matters

Feature scaling ensures that no single feature dominates the learning process simply because of its measurement units.

The problem in our dataset:

```
Area_sqft      : 200 - 12,000
Age            : 0   - 95
Distance_to_Metro : 0.1 - 50 (km)
Bedrooms       : 1   - 8
```

When you compute the **Euclidean distance** between two houses (for KNN) or take **gradient steps** (for Linear/Logistic Regression), Area_sqft contributes differences of thousands while Bedrooms contributes differences of 1–7. Area_sqft overwhelms the calculation completely.

5.2 Standardisation (Z-Score Normalisation)

Transforms each feature to have **mean = 0** and **standard deviation = 1**.

$$x_{\text{scaled}} = (x - \text{mean}(x)) / \text{std}(x)$$

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
df[['Area_sqft', 'Age', 'Distance_to_Metro']] = scaler.fit_transform(
    df[['Area_sqft', 'Age', 'Distance_to_Metro']]
)
```

After standardisation:

- **Area_sqft** → values roughly in range [-2, +2] (most between -3 and +3)
- **Age** → same range
- **Bedrooms** → same range

Now all features are on the same scale and contribute equally to distance/gradient calculations.

When to use: Default choice for most algorithms. Works well even with outliers (doesn't clip values). Required for Linear Regression (GD), Logistic Regression (GD), SVM, KNN, PCA.

Does NOT transform to a fixed range — if outliers exist, the range of scaled values can still be large.

5.3 Min-Max Normalisation

Transforms each feature to a fixed range, typically **[0, 1]**.

$$x_{\text{scaled}} = (x - x_{\text{min}}) / (x_{\text{max}} - x_{\text{min}})$$

```
from sklearn.preprocessing import MinMaxScaler

scaler = MinMaxScaler(feature_range=(0, 1))
df[['Area_sqft', 'Age']] = scaler.fit_transform(df[['Area_sqft', 'Age']])
```

After Min-Max:

- Minimum value → 0
- Maximum value → 1
- All other values proportionally between 0 and 1

When to use: When you need values in a specific bounded range. Neural networks often expect inputs in [0,1] or [-1,1]. Image pixel values are normalised to [0,1].

Weakness: Very sensitive to outliers. If **Age** has one outlier at 200 years (data error), the scaler maps the entire real data into a very narrow band [0, 0.475] while the outlier gets value 1.0 — distorting the entire feature.

5.4 Robust Scaler

Uses the **median and IQR** instead of mean and std — making it resistant to outliers.

$$x_scaled = (x - \text{median}(x)) / \text{IQR}(x)$$

```
from sklearn.preprocessing import RobustScaler

scaler = RobustScaler()
df[['Price', 'Area_sqft']] = scaler.fit_transform(df[['Price', 'Area_sqft']])
```

Why it's robust: Median and IQR are rank-based statistics. A single extreme value at Price=8 Crore does not change the median or IQR significantly, so the scaled values for the other 9,999 houses are not distorted.

When to use: When your data has outliers that you want to retain (real luxury houses) but don't want to distort the scaling for the majority.

5.5 When NOT to Scale

Model	Needs Scaling?	Reason
Linear Regression (GD)	✓ Yes	Gradient steps proportional to feature magnitude
Logistic Regression (GD)	✓ Yes	Same as above
KNN	✓ Yes	Distance calculations dominated by large-scale features
SVM	✓ Yes	Margin and kernel computations affected by scale
Neural Networks	✓ Yes	Activation functions and gradient flow need normalised inputs
Decision Tree	✗ No	Splits are based on ordering — scale doesn't change relative order
Random Forest	✗ No	Ensemble of decision trees
XGBoost / LightGBM	✗ No	Tree-based — scale invariant
Naive Bayes	✗ No	Works with probabilities — scale doesn't affect probability estimates

5.6 The Critical Rule: Fit on Train, Transform Both

```
# CORRECT
scaler = StandardScaler()
scaler.fit(X_train)           # Learn mean and std from training data ONLY
X_train_scaled = scaler.transform(X_train)
X_test_scaled = scaler.transform(X_test) # Apply SAME scaler to test data

# WRONG – Data Leakage
scaler.fit(X_test)           # Never fit on test data
X_test_scaled = scaler.fit_transform(X_test) # fit_transform on test = leakage
```

Why? In the real world, you deploy your model before seeing new data. The scaler must use only the statistics of training data — mean and std of the houses you trained on. If you fit on test data, you've used future information to preprocess past data — data leakage.

PART 6 — Handling Imbalanced Datasets

6.1 What Is Class Imbalance?

In our classification task (Expensive: Yes/No), suppose:

```
Not Expensive (0):  8,500 houses  (85%)
Expensive (1):      1,500 houses  (15%)
```

This is **class imbalance**. A model that predicts "Not Expensive" for every single house achieves **85% accuracy** — without learning anything. The minority class (Expensive=1) is what you usually care about most.

Severe imbalance is when one class has < 5–10% of the samples. Common in:

- Fraud detection (0.1% fraudulent transactions)
- Disease diagnosis (rare diseases)
- Churn prediction (5% users churn)
- Defect detection in manufacturing

6.2 Detecting Imbalance

```
print(df['Expensive'].value_counts())
print(df['Expensive'].value_counts(normalize=True))
# Output:
# 0      0.85
# 1      0.15
```

Impact on evaluation: Always check class distribution before choosing metrics. Accuracy is completely misleading for imbalanced data — use F1, Recall, AUC-ROC, or Precision-Recall AUC.

6.3 Strategy 1 — Oversampling the Minority Class

Add more samples of the minority class to balance the training set.

Random Oversampling

Simply duplicate random samples from the minority class.

```
from imblearn.over_sampling import RandomOverSampler

ros = RandomOverSampler(sampling_strategy=0.5, random_state=42)
# sampling_strategy=0.5 means minority will be 50% of majority after resampling
X_resampled, y_resampled = ros.fit_resample(X_train, y_train)
```

Problem: Exact duplicates of existing points → model can overfit to them.

SMOTE — Synthetic Minority Over-sampling Technique

Instead of duplicating, SMOTE **creates new synthetic samples** by interpolating between existing minority class samples.

```
Algorithm:
1. For each minority class sample x_i:
2. Find its K nearest neighbors (also minority class)
3. Randomly select one neighbor x_nn
4. Create a synthetic sample:
   x_synthetic = x_i + lambda * (x_nn - x_i)
   where lambda is a random number between 0 and 1
```

```
from imblearn.over_sampling import SMOTE

smote = SMOTE(sampling_strategy=0.5, k_neighbors=5, random_state=42)
X_resampled, y_resampled = smote.fit_resample(X_train, y_train)

print(f"Before SMOTE: {y_train.value_counts().to_dict()}")
print(f"After SMOTE: {pd.Series(y_resampled).value_counts().to_dict()}")
```

Why SMOTE is better than random oversampling: Synthetic samples are new points on the line segment between two real points — plausible new data rather than exact copies. The model doesn't just memorise duplicates; it learns the shape of the minority cluster.

Important: Apply SMOTE **only on training data**. Never on test data. Test data must reflect the real-world distribution.

6.4 Strategy 2 — Undersampling the Majority Class

Remove samples from the majority class to balance the dataset.

Random Undersampling

```
from imblearn.under_sampling import RandomUnderSampler

rus = RandomUnderSampler(sampling_strategy=0.5, random_state=42)
X_resampled, y_resampled = rus.fit_resample(X_train, y_train)
```

Problem: You discard potentially valuable majority class data. If your dataset is already small, this worsens it further.

When to use: When you have a very large dataset and can afford to discard majority class samples without losing statistical power.

Tomek Links

Remove majority class samples that are very close to minority class samples (the borderline "hard" cases).

```
from imblearn.under_sampling import TomekLinks

tl = TomekLinks()
X_clean, y_clean = tl.fit_resample(X_train, y_train)
```

Effect: Instead of random removal, this specifically removes the "noisy" majority samples near the decision boundary, making the boundary cleaner. Slight improvement in classifier performance.

6.5 Strategy 3 — Class Weights (Algorithm-Level Fix)

Instead of resampling, tell the algorithm to **penalise mistakes on the minority class more heavily** during training.

```
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC

# class_weight='balanced' automatically calculates weights
# inversely proportional to class frequency
model = LogisticRegression(class_weight='balanced')
model = SVC(class_weight='balanced')
```

```
# Or specify manually:  
# Minority class (1) is 5.67x rarer → penalise mistakes on it 5.67x more  
model = LogisticRegression(class_weight={0: 1, 1: 5.67})
```

Formula for balanced class weight:

$$w_j = n_{\text{samples}} / (n_{\text{classes}} * n_{\text{samples_in_class_j}})$$

Why this is often the cleanest solution: No data is created or discarded. The model itself learns to focus more on the minority class. Works seamlessly in cross-validation without resampling leakage.

6.6 Choosing the Right Strategy

```
Dataset is large (>100K rows)?  
└─ Undersampling is feasible → RandomUnderSampler or TomekLinks  
  
Dataset is small-medium?  
└─ Oversampling is safer → SMOTE preferred over RandomOverSampler  
  
Model supports class_weight parameter? (sklearn models do)  
└─ Try class_weight='balanced' first – cleanest, no data manipulation  
  
Imbalance ratio > 100:1 (extreme imbalance)?  
└─ Combine: SMOTE + Tomek Links (oversample minority, clean majority boundary)  
    from imblearn.combine import SMOTETomek
```

PART 7 — Feature Selection

7.1 Why Remove Features?

More features is not always better. Irrelevant or redundant features:

- Add noise that confuses the model
- Increase training time (especially for SVM, KNN)
- Risk of overfitting (model fits noise in irrelevant features)
- Make the model harder to interpret
- Worsen performance on new data (curse of dimensionality)

The goal: Retain only features that contribute genuine signal about the target.

7.2 Filter Methods (Model-Independent)

Correlation with Target

```
correlation = df.corr()['Price'].abs().sort_values(ascending=False)
print(correlation)

# Drop features with very low correlation with target
low_corr_features = correlation[correlation < 0.05].index
df.drop(columns=low_corr_features, inplace=True)
```

Limitation: Only detects linear relationships. A feature with near-zero correlation might still have a strong non-linear relationship with the target.

Correlation Between Features (Multicollinearity)

If two features are highly correlated with each other, one is redundant.

```
corr_matrix = df.corr().abs()

# Find pairs with correlation > 0.9
high_corr_pairs = []
for i in range(len(corr_matrix.columns)):
    for j in range(i):
        if corr_matrix.iloc[i, j] > 0.9:
            high_corr_pairs.append((corr_matrix.columns[i],
                                   corr_matrix.columns[j]))

print(high_corr_pairs)
# e.g., ('Area_sqft', 'Area_sqm') → same information, different units → drop one
```

Why multicollinearity is a problem for linear models: When two features are nearly identical, their coefficients become unstable — a tiny change in training data can flip one from +100 to -100 to compensate. The model becomes unreliable.

Variance Threshold

Remove features that have near-zero variance — they carry almost no information.

```
from sklearn.feature_selection import VarianceThreshold

selector = VarianceThreshold(threshold=0.01)
X_selected = selector.fit_transform(X)
```

Example: If 99.5% of all houses have `Has_Swimming_Pool = 0`, this feature is nearly constant and contributes almost nothing to discrimination.

7.3 Wrapper Methods

Recursive Feature Elimination (RFE)

Train a model, find the least important feature, remove it, retrain, repeat.

```
from sklearn.feature_selection import RFE
from sklearn.linear_model import LinearRegression

model = LinearRegression()
rfe = RFE(estimator=model, n_features_to_select=5)
rfe.fit(X_train, y_train)

print("Selected features:", X_train.columns[rfe.support_])
print("Feature ranking:", rfe.ranking_)
```

Advantage: Accounts for feature interactions — a feature might look unimportant alone but matter in combination with others.

Cost: Trains the model many times (once per removed feature) — expensive for large datasets.

7.4 Embedded Methods

Feature Importance from Tree-Based Models

Decision Trees and ensemble models (Random Forest, XGBoost) compute feature importance as part of training — no extra computation needed.

```
from sklearn.ensemble import RandomForestRegressor

rf = RandomForestRegressor(n_estimators=100, random_state=42)
rf.fit(X_train, y_train)

importances = pd.Series(rf.feature_importances_, index=X_train.columns)
importances.sort_values(ascending=False).plot(kind='bar')
plt.title('Feature Importance')
plt.show()

# Keep top 10 features
top_features = importances.nlargest(10).index
X_selected = X_train[top_features]
```

Importance score meaning: The fraction of the total information gain across all trees that is attributable to each feature. Higher = more important.

L1 Regularisation (Lasso) — Built-in Feature Selection

Lasso regression adds an L1 penalty that drives coefficients of unimportant features to **exactly zero** — effectively selecting features automatically.

$$J_{\text{lasso}} = \text{MSE} + \lambda * \text{Sum}(|w_i|) \quad \leftarrow \text{L1 penalty}$$

```
from sklearn.linear_model import Lasso

lasso = Lasso(alpha=0.01)
lasso.fit(X_train_scaled, y_train)

# Features with non-zero coefficients are selected
selected = X_train.columns[lasso.coef_ != 0]
print("Selected features:", list(selected))
print("Zeroed out features:", list(X_train.columns[lasso.coef_ == 0]))
```

Why L1 zeroes out coefficients (but L2 doesn't): The L1 penalty has a "corner" at zero in the optimisation geometry. Gradient descent solutions land exactly at zero for unimportant features. L2 (Ridge) shrinks coefficients toward zero but rarely to exactly zero.

PART 8 — Feature Engineering

8.1 What Is Feature Engineering?

Feature engineering is the process of **creating new features from existing ones** to better capture the underlying patterns in the data. It is where domain knowledge meets mathematics.

A model can only learn from the features you give it. If the true predictor of Price is **Price_per_sqft** but you only give the model **Price** and **Area** separately, the model must learn this ratio itself — which it may or may not do well depending on the algorithm and amount of data. Feature engineering makes implicit relationships **explicit** for the model.

8.2 Interaction Features

Combine two features whose joint effect is more predictive than either alone.

```
# Price per square foot – a fundamental real estate metric
df['Price_per_sqft'] = df['Price'] / df['Area_sqft']

# Total room density
df['Area_per_room'] = df['Area_sqft'] / df['Bedrooms']

# Is the house large AND has many bedrooms? (multiplicative interaction)
df['Area_x_Bedrooms'] = df['Area_sqft'] * df['Bedrooms']
```

Why this matters: Linear Regression can only learn additive effects $w_1 \text{Area} + w_2 \text{Bedrooms}$. It cannot learn multiplicative effects $\text{Area} * \text{Bedrooms}$ unless you explicitly provide that product as a feature. Tree-based models can learn interactions themselves, but providing them explicitly speeds up learning.

8.3 Polynomial Features

Create squared, cubed, or higher-order versions of features to capture non-linear relationships.

```
from sklearn.preprocessing import PolynomialFeatures

poly = PolynomialFeatures(degree=2, include_bias=False)
X_poly = poly.fit_transform(X[['Area_sqft', 'Age']])

# From 2 features, this creates:
# Area, Age, Area2, Age2, Area*Age
# (2 original + 3 new = 5 total features for degree=2)
```

When to use: When you know or suspect a non-linear relationship. For example, **Price vs. Age** is likely U-shaped — very new buildings and very old (heritage) buildings are expensive, middle-aged buildings are cheaper. A squared term Age^2 captures this.

Warning: With many features, polynomial expansion explodes dimensionality. With 10 features at degree 2, you get 66 features. At degree 3 you get 286. Use with care.

8.4 Datetime Feature Extraction

Never feed raw datetime strings to a model. Extract meaningful components.

```
df['Last_Renovated'] = pd.to_datetime(df['Last_Renovated'])

# Extract components
df['Renovation_Year'] = df['Last_Renovated'].dt.year
df['Renovation_Month'] = df['Last_Renovated'].dt.month
df['Renovation_Quarter'] = df['Last_Renovated'].dt.quarter

# Calculate time since renovation (more useful than absolute year)
```

```
df['Years_Since_Renovation'] = 2024 - df['Renovation_Year']

# Was it recently renovated? (binary flag)
df['Recently_Renovated'] = (df['Years_Since_Renovation'] <= 5).astype(int)

# Season of listing (summer vs. winter demand cycles)
df['Listing_Season'] = df['Last_Renovated'].dt.month.map({
    12: 'Winter', 1: 'Winter', 2: 'Winter',
    3: 'Spring', 4: 'Spring', 5: 'Spring',
    6: 'Summer', 7: 'Summer', 8: 'Summer',
    9: 'Autumn', 10: 'Autumn', 11: 'Autumn'
})
```

8.5 Binning / Discretisation

Convert continuous features into categorical buckets when the relationship with the target is non-linear and step-wise.

```
# Equal-width bins
df['Age_Group'] = pd.cut(df['Age'],
    bins=[0, 5, 15, 30, 50, 100],
    labels=['New', 'Recent', 'Middle-aged', 'Old', 'Heritage']
)

# Equal-frequency bins (same number of houses in each bin)
df['Price_Quartile'] = pd.qcut(df['Price'], q=4,
    labels=['Budget', 'Mid-range', 'Premium', 'Luxury']
)

# Custom business-driven bins
df['Metro_Accessibility'] = pd.cut(df['Distance_to_Metro'],
    bins=[0, 0.5, 2, 5, 100],
    labels=['Walking', 'Nearby', 'Moderate', 'Far']
)
```

When to use: When a numerical feature has a threshold effect. For example, being 0.5 km vs. 0.6 km from the metro might not matter much, but 0.5 km vs. 5 km is a huge lifestyle difference. Binning captures this step-wise behaviour.

8.6 Aggregation Features (Group Statistics)

Create features that summarise information about a group that each row belongs to.

```
# For each location, compute statistics of all houses in that location
location_stats = df.groupby('Location').agg(
    Location_Avg_Price = ('Price', 'mean'),
    Location_Median_Price = ('Price', 'median'),
)
```

```

    Location_Avg_Area      = ('Area_sqft', 'mean'),
    Location_Count         = ('Price', 'count')      # how many houses in this
area
).reset_index()

df = df.merge(location_stats, on='Location', how='left')

```

Why these are powerful: The average price in a location captures the neighbourhood desirability signal that raw location encoding misses. A house in a location where the average price is 80 Lakhs is very different from one where the average is 2 Crore.

Important: Compute these statistics on **training data only**, then apply to test data. Computing on the full dataset leaks test information.

8.7 Text Feature Engineering (Brief)

If the dataset included a **Description** column (free text about the house):

```

from sklearn.feature_extraction.text import TfidfVectorizer

# TF-IDF: how important is each word in this description
# relative to all other descriptions
tfidf = TfidfVectorizer(max_features=100, stop_words='english')
desc_features = tfidf.fit_transform(df['Description'])

# Or create simple keyword flags
df['Has_Garden'] = df['Description'].str.contains('garden|terrace',
case=False).astype(int)
df['Has_Parking'] = df['Description'].str.contains('parking|garage',
case=False).astype(int)
df['Has_Sea_View'] = df['Description'].str.contains('sea view|ocean',
case=False).astype(int)

```

8.8 Log and Power Transformations as Feature Engineering

Beyond outlier handling, transformations fundamentally reshape the relationship between a feature and the target.

```

# Right-skewed features → log transform
df['Log_Area'] = np.log1p(df['Area_sqft'])
df['Log_Price'] = np.log1p(df['Price'])

# Check before and after
print(f"Area skewness before: {df['Area_sqft'].skew():.2f}")
print(f"Area skewness after: {df['Log_Area'].skew():.2f}")

```

```
# Before: 3.4 (strongly right-skewed)
# After: 0.2 (approximately normal)
```

The Box-Cox Transformation is the general power transformation that finds the optimal lambda to normalise any feature:

```
y_transformed = (y^lambda - 1) / lambda    if lambda ≠ 0
               = log(y)                    if lambda = 0
```

```
from scipy.stats import boxcox
from sklearn.preprocessing import PowerTransformer

pt = PowerTransformer(method='box-cox')    # requires all positive values
pt = PowerTransformer(method='yeo-johnson') # works with negative values too
df['Area_transformed'] = pt.fit_transform(df[['Area_sqft']])
```

PART 9 — Handling Skewed Distributions

9.1 Why Skewness Matters

Linear Regression assumes the residuals (errors) are normally distributed. If the target variable **Price** is heavily right-skewed, this assumption is violated and the model performs poorly.

Skewness measure:

```
Skewness = (1/n) * Sum[ ((x_i - mean) / std)3 ]
```

```
Skewness ≈ 0    : Symmetric (normal-ish)
Skewness > 1    : Right-skewed (long right tail)
Skewness < -1   : Left-skewed (long left tail)
```

```
print(df['Price'].skew())    # e.g., 4.2 → strongly right-skewed

# Visualise
import scipy.stats as stats
stats.probplot(df['Price'], dist='norm', plot=plt)    # QQ-plot
plt.title('QQ Plot of Price - Before Transformation')
plt.show()
```

Fix: Log-transform the target before training. Predict log(Price), then exponentiate to get Price.

PART 10 — The Complete Preprocessing Pipeline

10.1 Why Use a Pipeline?

A **Pipeline** chains all preprocessing steps and the model into a single object. This is not just convenient — it is **essential for correctness**.

The leakage danger without pipelines:

```
# WRONG – data leakage
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)          # fitted on ALL data including test
X_train, X_test = train_test_split(X_scaled, ...) # test already seen by scaler
```

With a pipeline:

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder
from sklearn.impute import SimpleImputer

# Define which columns are numerical and which are categorical
numerical_cols = ['Area_sqft', 'Age', 'Distance_to_Metro', 'Bedrooms']
categorical_cols = ['Location', 'Furnishing']

# Preprocessing for numerical columns
numerical_pipeline = Pipeline([
    ('imputer', SimpleImputer(strategy='median')),
    ('scaler', StandardScaler())
])

# Preprocessing for categorical columns
categorical_pipeline = Pipeline([
    ('imputer', SimpleImputer(strategy='most_frequent')),
    ('encoder', OneHotEncoder(drop='first', handle_unknown='ignore'))
])

# Combine both pipelines
preprocessor = ColumnTransformer([
    ('num', numerical_pipeline, numerical_cols),
    ('cat', categorical_pipeline, categorical_cols)
])

# Full pipeline including the model
full_pipeline = Pipeline([
    ('preprocessor', preprocessor),
    ('model', LogisticRegression(C=1.0, class_weight='balanced'))
])
```

```
])

# Now fit ONLY on training data – scaler, imputer, encoder all fit on train only
full_pipeline.fit(X_train, y_train)

# Transform and predict – test data transformed using train statistics
y_pred = full_pipeline.predict(X_test)
```

The pipeline automatically:

- Fits all transformers on training data only
- Applies those same fitted transformers to test data
- Combines everything into a single `.fit()` and `.predict()` call
- Makes cross-validation correct (each fold fits its own scaler on its fold's training data)

10.2 Complete Preprocessing Checklist

PHASE 1: UNDERSTAND

- ☐ Check shape, dtypes, `df.info()`, `df.describe()`
- ☐ Check missing value counts and percentages
- ☐ Visualise distributions (histograms, box plots)
- ☐ Check class distribution (for classification)
- ☐ Identify data types: continuous, discrete, nominal, ordinal, datetime, text

PHASE 2: CLEAN

- ☐ Drop rows where TARGET variable is missing
- ☐ Investigate WHY values are missing (MCAR/MAR/MNAR)
- ☐ Handle missing values (impute or drop) – fit on train only
- ☐ Detect and handle outliers (Z-score or IQR)
- ☐ Fix data entry errors (Bedrooms=18 → 1 or 8)
- ☐ Remove duplicate rows

PHASE 3: ENCODE

- ☐ Encode ordinal categories (OrdinalEncoder with correct order)
- ☐ Encode nominal categories (OHE for low cardinality, Target Encoding for high)
- ☐ Handle datetime columns (extract year, month, days_since, etc.)

PHASE 4: SCALE

- ☐ Check which model you are using (tree-based → no scaling needed)
- ☐ Choose scaler (Standard for most, Robust if outliers, MinMax for neural nets)
- ☐ Fit scaler on training data only, transform both train and test

PHASE 5: ENGINEER FEATURES

- ☐ Create interaction features (Area * Bedrooms, Price_per_sqft)
- ☐ Create aggregation features (location averages)
- ☐ Apply log/power transforms to skewed features
- ☐ Bin continuous features where step-wise relationship exists
- ☐ Extract datetime components

PHASE 6: SELECT FEATURES

- Remove near-zero variance features
- Remove highly correlated features (correlation > 0.9)
- Use model-based importance (Random Forest, Lasso) to rank features
- Remove features with very low correlation to target

PHASE 7: HANDLE IMBALANCE (if classification)

- Check class distribution
- Apply SMOTE or `class_weight='balanced'` (train data only)

PHASE 8: WRAP IN PIPELINE

- Build sklearn Pipeline or ColumnTransformer
- Ensure `fit_transform` is called only on training data
- Use `pipeline.fit(X_train)`, `pipeline.predict(X_test)`

PART 11 — Interview Answer Bank

 These are the exact questions asked in ML interviews regarding preprocessing. Read each answer once before an interview.

Q: What are the three types of missing data, and why does it matter which type you have?

MCAR (Missing Completely At Random) — missingness is pure chance, safe to impute or drop. MAR (Missing At Random) — missingness depends on other observed features, use KNN or iterative imputation which leverages those relationships. MNAR (Missing Not At Random) — the missing value is related to the value itself (wealthy owners hide income), dangerous to blindly impute — create a "was_missing" binary flag first. The type determines whether your imputation will introduce bias.

Q: Why must you fit the imputer/scaler on training data only?

In production, you deploy before seeing new data. The scaler and imputer must use only what was known during training. Fitting on test data means using future information to preprocess past data — this is data leakage. The reported test performance will be optimistically inflated and won't reflect real deployment performance.

Q: What is the difference between StandardScaler and MinMaxScaler? When do you use each?

StandardScaler transforms to mean=0, std=1 — it is robust to outliers and does not constrain the output range. MinMaxScaler transforms to [0,1] — it is simple and bounded but sensitive to outliers (one extreme value distorts the entire range). Use StandardScaler as the default for most ML algorithms. Use MinMaxScaler for neural networks expecting inputs in [0,1] or [-1,1]. Use RobustScaler when you have meaningful outliers you want to retain without distortion.

Q: Why is One-Hot Encoding needed for nominal categories? Why not just Label Encode?

Label encoding assigns integers (0, 1, 2...) which imply an ordinal relationship. A linear model would interpret "Unfurnished=2" as mathematically greater than "Furnished=0" — a completely false relationship. One-Hot Encoding creates independent binary columns for each category, allowing the model to learn a separate weight for each without implying any ordering.

Q: What is the dummy variable trap?

If a categorical column has K categories and you create K one-hot columns (not K-1), perfect multicollinearity exists — one column is always perfectly predictable from the others. For example, if `Furnished=0` and `Semi-Furnished=0`, then `Unfurnished` must be 1. This redundancy causes the normal equations in linear regression to be non-invertible. Fix: always use `drop_first=True` in OHE.

Q: What is SMOTE and how is it different from random oversampling?

SMOTE (Synthetic Minority Over-sampling Technique) creates new synthetic minority class samples by interpolating between existing ones. For each minority sample, it finds K nearest minority neighbors and creates a synthetic point on the line segment between them. Random oversampling simply duplicates existing samples. SMOTE is better because the model sees new, plausible data rather than exact copies, reducing the risk of overfitting to duplicated points.

Q: You have a feature with 40% missing values. What do you do?

First investigate the pattern: is it MCAR, MAR, or MNAR? Check if the missing values correlate with other features or with the target. If MNAR, create a `feature_missing` binary flag first. For 40% missing: simple imputation will distort the distribution significantly — prefer KNN or iterative (MICE) imputation if computational cost allows. Consider whether the feature has enough non-missing signal to justify keeping it at all — if the remaining 60% has low correlation with the target, dropping the column might be the right call. Never drop 4,000 rows from a 10,000-row dataset just for one feature's missing values.

Q: What is feature engineering vs. feature selection?

Feature engineering is **creating new features** from existing ones to make implicit patterns explicit — like computing `Price_per_sqft` from `Price` and `Area`. Feature selection is **choosing which existing features** to keep and which to discard. Both serve the same goal (giving the model the most useful signal) but at different stages: engineer first, then select.

Q: Why can a high-cardinality categorical column (like postal code with 10,000 unique values) break One-Hot Encoding?

OHE with 10,000 unique values creates 9,999 new columns. Most are nearly all zeros (sparse). Rare postal codes appear in only 1-2 rows — their OHE columns are almost completely uninformative and purely noise. Dimensionality explodes, making the model slower, more prone to overfitting, and harder to train. Use target encoding (mean price per postal code with smoothing) or frequency encoding to collapse this to 1 column instead.

Q: What is the difference between L1 (Lasso) and L2 (Ridge) regularisation for feature selection?

L1 penalty is $\lambda \sum |w_i|$ — it drives unimportant feature weights to **exactly zero**, performing automatic feature selection. L2 penalty is $\lambda \sum w_i^2$ — it shrinks all weights toward zero but rarely reaches exactly zero; all features are retained with small weights. L1 is preferred when you believe many features are truly irrelevant. L2 is preferred when all features are somewhat relevant but you want to prevent any from dominating.

Q: What is multicollinearity and why is it a problem?

Multicollinearity is when two or more features are highly correlated with each other. In linear models, it makes the coefficient estimates unstable — a tiny change in training data can flip a coefficient from +500 to -500 while another compensates. The model loses interpretability and generalisation. Detect with a correlation matrix (threshold > 0.9) or Variance Inflation Factor (VIF > 5–10). Fix by removing one of the correlated features, using PCA to combine them, or applying Ridge regularisation which handles multicollinearity gracefully.

Q: Walk me through your complete preprocessing pipeline on a new dataset.

First, understand the data: shape, dtypes, missing values, distributions, class balance. Then clean: drop rows where the target is missing, investigate missingness type, impute appropriately, handle outliers by capping or log-transforming skewed features. Encode: ordinal categories with OrdinalEncoder specifying the correct order, nominal low-cardinality categories with OHE (drop_first), high-cardinality with target encoding on training data only. Scale: StandardScaler fit on train, applied to both train and test. Engineer features: interaction terms, datetime components, aggregation statistics. Select: remove near-zero variance, drop one feature from highly correlated pairs, use tree-based importance or Lasso to rank remaining features. Handle imbalance if applicable: SMOTE on training data or class_weight='balanced'. Wrap everything in a sklearn Pipeline so that all fitting happens on training data only, preventing any leakage.

The one truth that unifies everything in data preprocessing: *Every transformation you apply is based on statistics learned from the training data. The moment you let your preprocessing see the test data before the final evaluation, you have contaminated your experiment. The Pipeline is not just a convenience — it is your guarantee of scientific honesty.*

"Raw data is like an unpolished diamond. Preprocessing is the cutting and polishing. No matter how brilliant the jeweller (your algorithm), a poorly cut stone will never shine as brightly as a well-prepared one."